SECTION 1 · SUBSECTION 1.1
BHATKHANDE-FLOW RENDERING ENGINE

1.1.1 INTRODUCTION.

Bhatkhande-Flow is a rendering engine for Bhatkhande notation. It reads a bandish as JSON, swartaal XML or a Label Studio export and draws the printed page as SVG. Every drawn element carries a BhatkhandeSet class name, so one run also yields training labels (figure 1.1-1). The pipeline is VexFlow's: measure first, place second, draw through an abstract context.

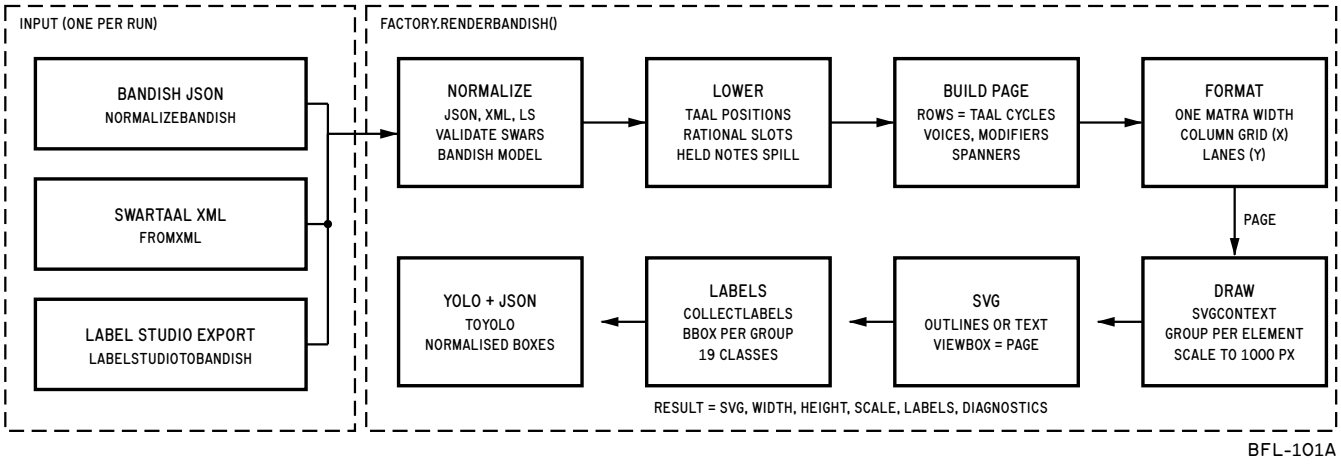


Figure 1.1-1. Rendering Pipeline Functional Diagram

1.1.2 FUNCTIONAL DESCRIPTION.

Factory.renderBandish() sets the style and fonts, parses the input, lowers it to rational slots, builds one row per taal cycle and formats the whole page. Page.draw() then draws the header, the rows and the spanners into an SVGContext, which records a bounding box for every element group. Source size by module is shown in figure 1.1-2.

1.1.2.1 One Page, One Width.

Every row is measured first. The page then takes one matra width: the widest packed matra, or 1.25 times the swar size if larger, plus 7 px of padding each side. A page wider than 1000 px is scaled down as a whole. Rows never wrap.

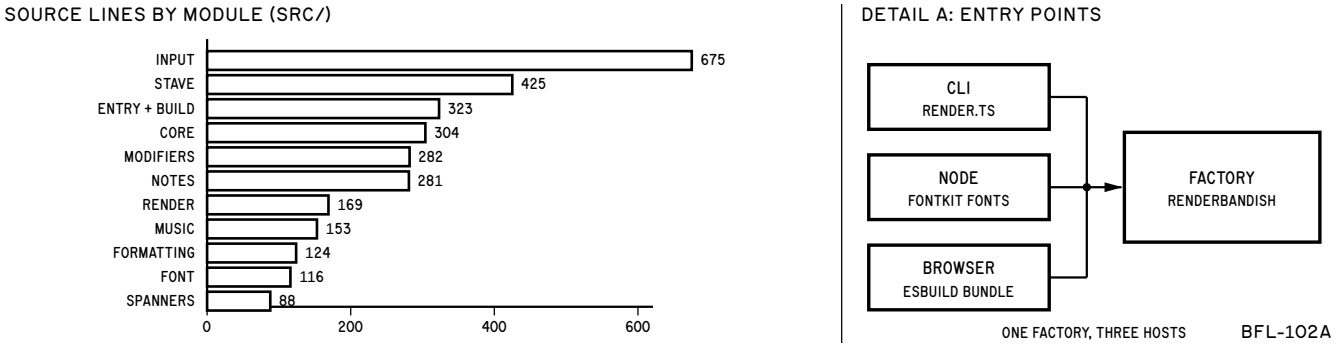


Figure 1.1-2. Source Modules And Entry Points

1.1.3 PERFORMANCE AND DESIGN DATA.

Table 1.1-1. Engine Data

Parameter	Value
Source files (TypeScript)	53 FILES, 2,940 LINES
Test suite	18 TESTS, ALL PASS
Taal definitions	11
Label classes	19
Example bandish	7 JSON, 7 LABEL STUDIO
Page width limit	1000 PX, THEN SCALED
Swar / lyric font size	24 / 21 PX
Output per render	SVG, LABELS, YOLO (+ PNG)

CAUTION

Run *npm install* on the machine that runs the code. A node_modules folder built on Linux carries the wrong esbuild binary,.

RENDER A BANDISH — USER

```
1 SHELL - cd bhatkhande-flow
2 SHELL - npm install
3 SHELL - npm test
  √ tests - 18 pass
4 SHELL - npm run render --
    examples/bhimpalasi-geet.json
    out --png
  √ out - svg, labels, yolo, png
```

SECTION 1 · SUBSECTION 1.2
VEXFLOW ANALYSIS AND CLASS MAPPING

1.2.1 INTRODUCTION.

The engine copies VexFlow's mechanism wherever the problem is the same, and follows the printed Bhatkhande books wherever it differs. Four ideas carry over: elements report their widths before any x is assigned; contexts resolve collisions; the formatter owns x and the stave owns y; spanners are drawn last, from final positions (figure 1.2-1).

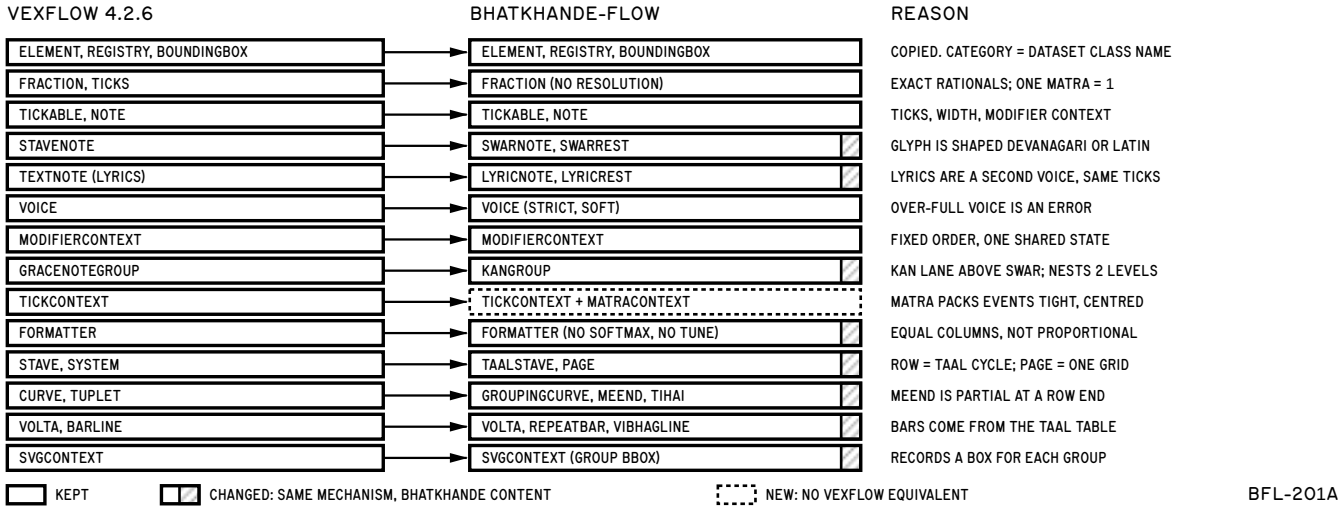


Figure 1.2-1. VexFlow Class Mapping

1.2.2 FUNCTIONAL DESCRIPTION.

Notes and modifiers join one ModifierContext per onset, then one TickContext per onset across the swar and lyric voices. A MatraContext groups the TickContexts of one matra, packs them tightly and centres the cluster in its column. TaalStave stacks the lanes; Page gives every column its x from one grid (figure 1.2-2).

1.2.2.1 Stacking Order.

preFormat() calls each modifier class once, inner to outer. All five share one state, so a class stacks outside what came before it: komal bar below the glyph, then mandra dots; tivra stroke above, then taar dots, then the kan lane.

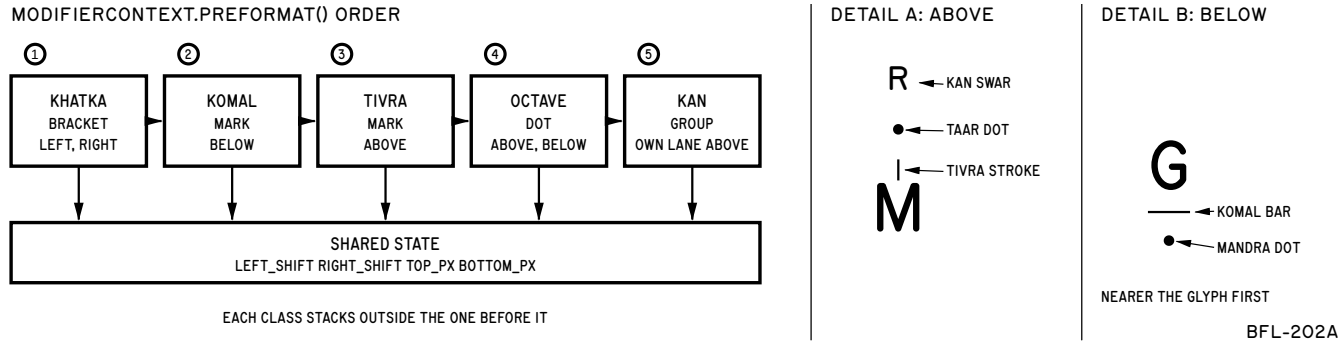


Figure 1.2-2. Modifier Stacking Order And Detail

1.2.3 PERFORMANCE AND DESIGN DATA.

Table 1.2-1. Corrections To The First Engine

Defect in v0.x	Rule in v1
Taal markers above the swars	BELOW THE LYRICS
Held swar written as avagraha	DASH IN SWAR LINE
Bar at the start of a row	NO OPENING BAR
Rows broken by page width	ROW = ONE TAAL CYCLE
Sub-beats spread apart	PACKED, CURVE UNDER
Matra width varies by row	ONE WIDTH PER PAGE
Kan as a small superscript	OWN LANE ABOVE SWAR
Parentheses read as murki	KHATKA BRACKETS
Grouping drawn as a bracket	CURVE UNDER SWARS

NOTE

Copy the mechanism when the problem is the same. When Bhatkhande differs, the scanned page decides, not Western habit.

ADD A MODIFIER — DEVELOPER

```
1 src/modifiers - new class
2 CATEGORY - dataset class name
3 static format(members, state)
4 ModifierContext - call in order
5 builder.decorate - attach it
6 labels.ts - add to classes
7 test - add a case
```

BFL

SECTION 1 · SUBSECTION 1.3
NOTATION CONVENTIONS

1.3.1 INTRODUCTION.

A printed Bhatkhande page is a table. One row is one taal cycle, every matra column has the same width, and the vibhag bars line up from row to row (figure 1.3-1). The conventions were checked on the scanned books, not taken from memory, and the main ones are pinned by tests.

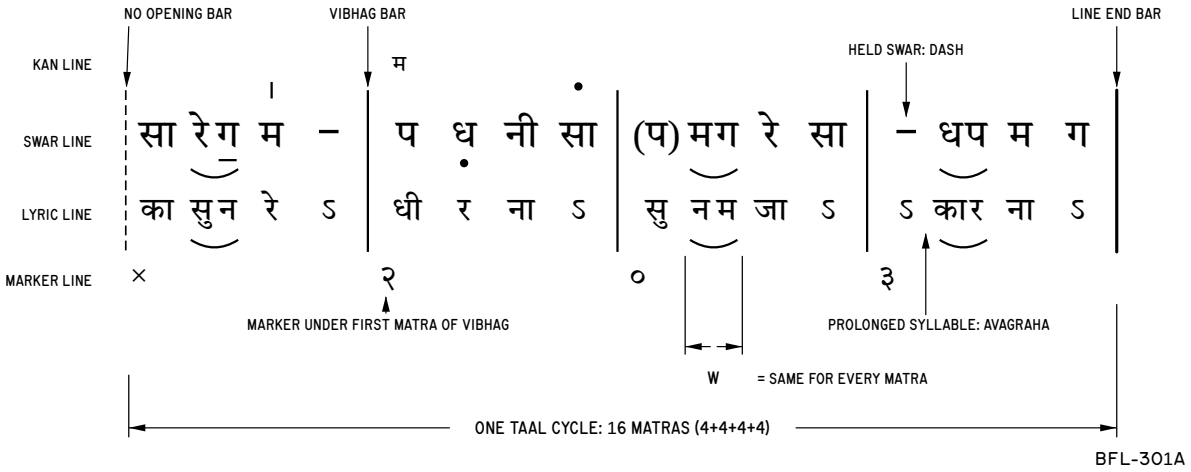


Figure 1.3-1. Printed Row Of Teentaal, Lane By Lane

1.3.2 FUNCTIONAL DESCRIPTION.

Lines run top to bottom: kan, swar, lyric, then the taal marker under the first matra of each vibhag. A held swar is a dash in the swar line and an avagraha in the lyric line. Swars that share one matra are written close together with a curve under them. The marks are shown in figure 1.3-2.

1.3.2.1 Pickup Rows.

A row that starts mid-cycle sits at its cycle position, so it is right-aligned and closed by a bar. The last row may stop short and is left as it is.

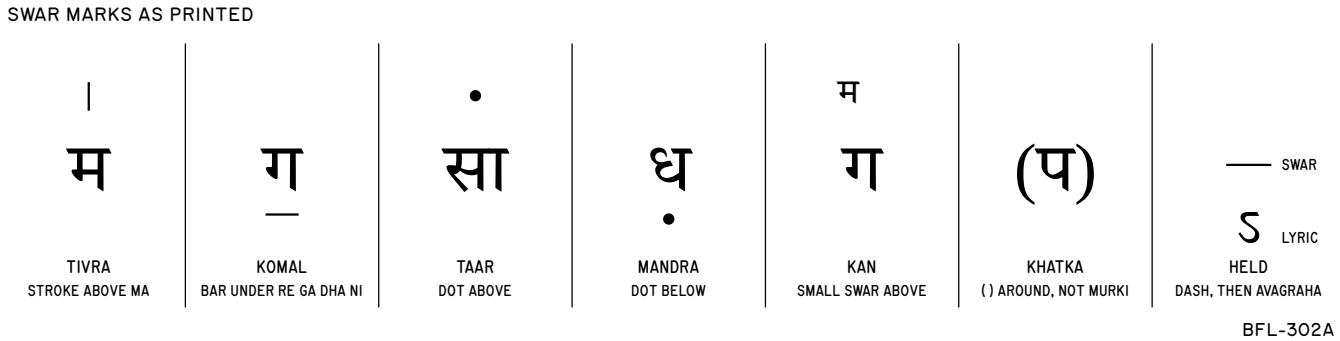


Figure 1.3-2. Swar Marks

1.3.3 PERFORMANCE AND DESIGN DATA.

Table 1.3-1. Printed Conventions Verified On The Scans

Convention	Printed rule
B1 Row	ONE TAAL CYCLE
B2 Line order	KAN, SWAR, LYRIC, MARKERS
B3 Bars	BETWEEN VIBHAGS AND AT ROW END
B4 Held swar	DASH; AVAGRAHA IN LYRIC
B5 Grouped swars	CURVE UNDER SWAR AND LYRIC
B6 Matra column	SAME WIDTH ON THE PAGE
B7 Pickup	AT CYCLE POSITION, RIGHT-ALIGNED
B8 Last row	MAY STOP MID-CYCLE
B9 Marks	KOMAL UNDER; TIVRA, TAAR OVER
B10 Kan	SMALL SWAR ABOVE, LEFT EDGE
B11 Khatka	SWAR IN PARENTHESES

NOTE

A swar in parentheses is a khatka, not a murki. The engine draws two khatka brackets around the swar (B11).

AUTHOR A ROW	USER
1 TAAL - set metadata.taal.name	
2 MATRA - one object per matra	
3 EVENTS - swars in order	
4 HELD - use "-" for a held swar	
5 LYRIC - one syllable per swar	
√ MARKERS - derived from taal	

BFL

SECTION 1 · SUBSECTION 1.4
INPUT, NORMALIZATION AND LOWERING

1.4.1 INTRODUCTION.

Input is normalised into one BandishModel, whichever form it arrives in. Lowering then turns the model into a grid the layout can use. VexFlow draws a half note as one glyph, but Bhatkhande notation writes the time out: a note held three beats becomes a swar and two dashes (figure 1.4-1).

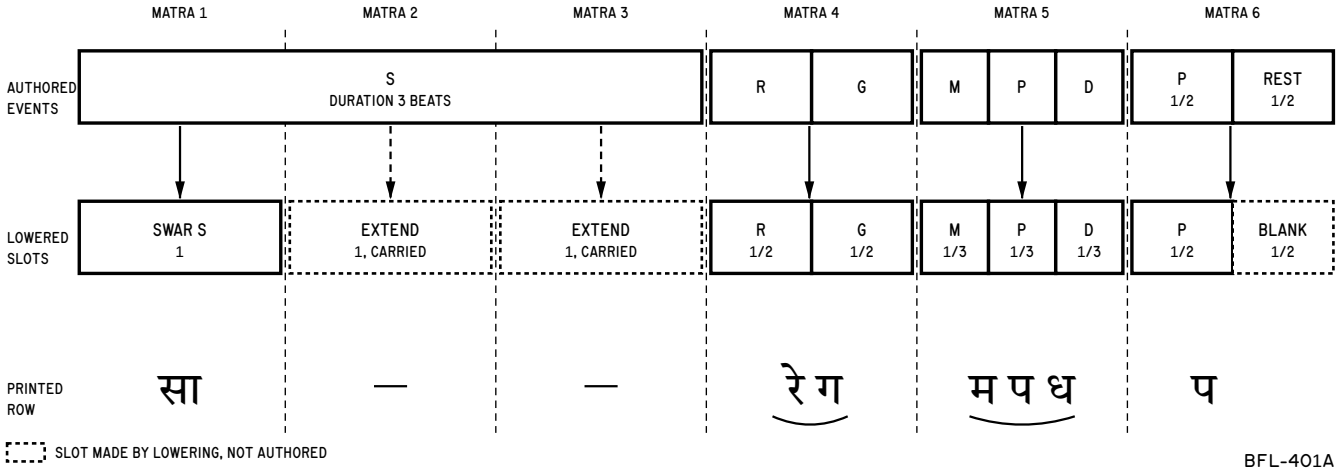


Figure 1.4-1. Lowering Of Durations Into Slots

1.4.2 FUNCTIONAL DESCRIPTION.

lowerBandish() resolves the taal, walks every matra and builds ordered slots whose durations sum to exactly one beat. Events with no duration share the rest of the matra equally. A duration longer than the room left spills into the next matras as carried slots. Taal position, vibhag and marker come from the taal table (figure 1.4-2).

1.4.2.1 Diagnostics.

A musical problem never stops the render. Each one is returned with a level, a code, a message and a path. An error marks inconsistent input; a warning marks input that was padded or ignored.

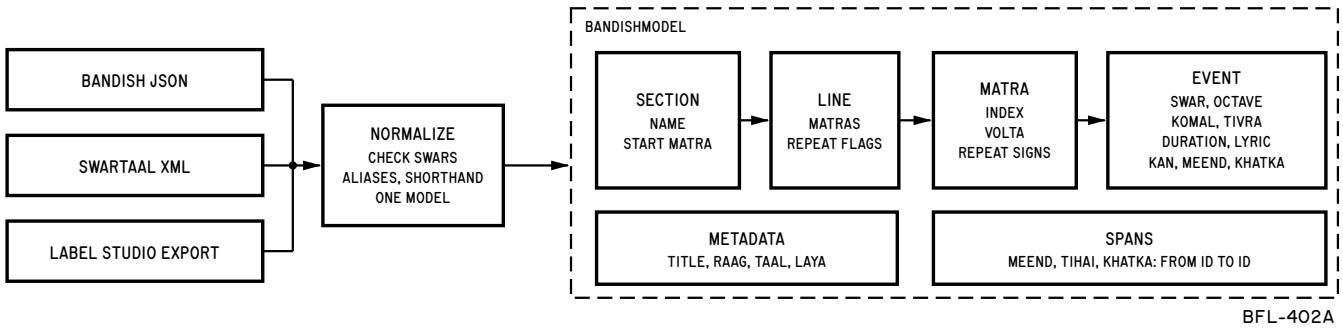


Figure 1.4-2. Input Forms And The Bandish Model

1.4.3 PERFORMANCE AND DESIGN DATA.

Table 1.4-1. Diagnostic Codes

Code	Level and meaning
matra.sequence	WARN: INDEX NOT THE EXPECTED POSITION
matra.underfull	WARN: SHORT OF 1 BEAT, PADDED
duration.overfull	ERROR: DURATIONS ALREADY FILL THE MATRA
carry.overlap	ERROR: HELD NOTE COVERS THE EVENTS
carry.dangling	WARN: SECTION ENDS WHILE HELD
meend.unclosed	WARN: MEEND NEVER CLOSED
tihai.sam	WARN: LAST NOTE MISSES SAM
repeat.offbar	WARN: SIGN NOT ON A VIBHAG BAR
marker.mismatch	WARN: MARKER DIFFERS FROM THE TAAL
matra.overflow	ERROR: EVENT AFTER MATRA IS FULL
rest.blank	INFO: SILENCE PRINTED AS EMPTY SLOT

CAUTION

Bhatkhande has no printed rest sign. A silence is drawn as an empty slot and reported as rest.blank, so a reader cannot tell it from a missing note.

CONVERT AN ANNOTATION USER

```
1 CONVERT - npx tsx
    cli/ls2bandish.ts out.json
    "TITLE" page1.json page2.json
2 RENDER - npm run render --
    out.json out/ls --png
    --no-lyric-curves
  √ compare - render beside scan
```

SECTION 1 · SUBSECTION 1.5
PAGE LAYOUT: COLUMN GRID AND LANES

1.5.1 INTRODUCTION.

The page is laid out as a grid. Every row is measured first, then one matra width is chosen for the whole page, so a given cycle position sits in the same column in every row (figure 1.5-1). A pickup row occupies the columns of its own cycle positions and so ends up right-aligned.

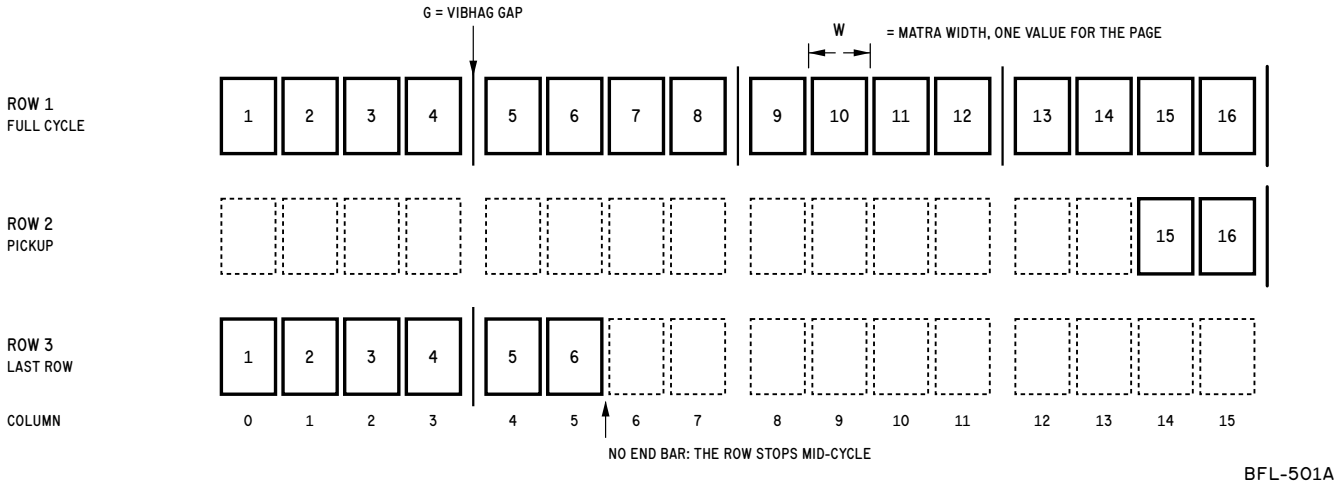


Figure 1.5-1. Page Column Grid, Three Kinds Of Row

1.5.2 FUNCTIONAL DESCRIPTION.

TaalStave.preFormat() builds the contexts of a row and reports its widest packed matra. Page.format() fixes the matra width, assigns x from the ColumnGrid, reserves meend and tihai lanes on the rows they touch, then stacks the lanes of each row top to bottom (figure 1.5-2).

1.5.2.1 Column Position.

x(col) = left + col × W + bars × G, where bars counts the vibhag bars crossed. A bar stands before a column only when that column starts a vibhag and is not the first in its row.

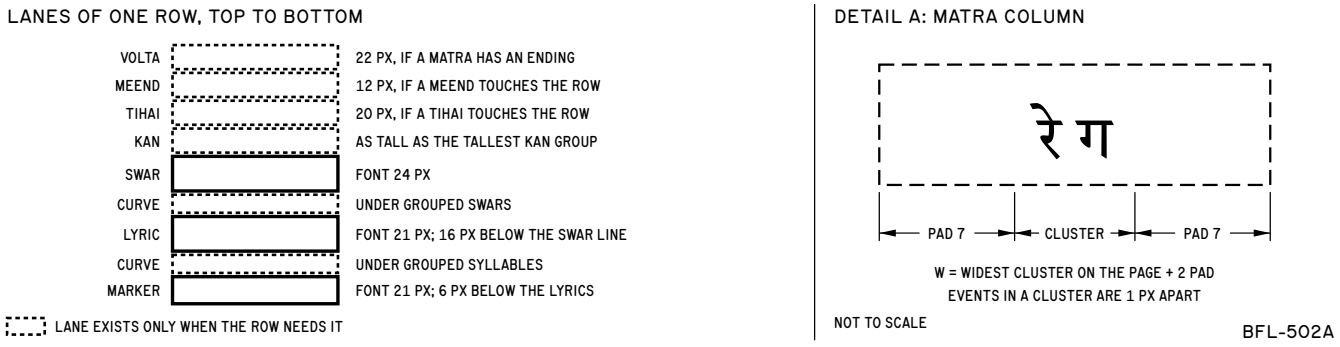


Figure 1.5-2. Row Lanes And Matra Column

1.5.3 PERFORMANCE AND DESIGN DATA.

Table 1.5-1. Default Layout Parameters

Style key	Default
spacing.matraPad	7 PX
spacing.clusterGap	1 PX
spacing.vibhagGap	16 PX
spacing.swarToLyric	16 PX
spacing.lyricToMarker	6 PX
spacing.rowGap	30 PX
spacing.margin	44 PX
spacing.maxWidth	1000 PX
spacing.sectionGap	14 PX
fontSize.swar / lyric	24 / 21 PX
fontSize.kanScale	0.6 OF SWAR

NOTE

Rows never wrap. A page wider than maxWidth is scaled down as a whole (scale = maxWidth / width), so the bars still line up.

```
TUNE THE LOOK                    DEVELOPER
1 FACTORY - pass options.style
2 spacing.vibhagGap - bar room
3 spacing.maxWidth - page scale
4 fontSize.swar - swar size
5 script - devanagari | latin
6 textMode - path | text
  √ render - bars still align
```

SECTION 1 · SUBSECTION 1.6
SPANNERS, ENDINGS AND REPEATS

1.6.1 INTRODUCTION.

Spanners join several notes and are drawn after the notes, from their final positions. Volta brackets, repeat signs, tihai brackets and meends follow VexFlow's Volta, Barline, Tuplet and Curve, and a spanner that crosses a row break is drawn as one piece per row (figure 1.6-1).

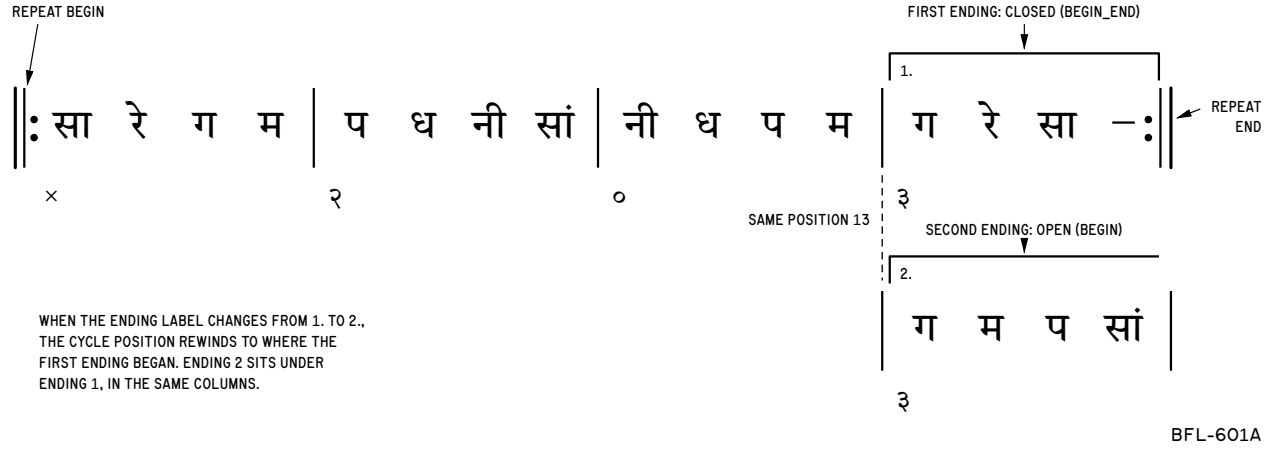


Figure 1.6-1. Alternate Endings And Repeat Signs

1.6.2 FUNCTIONAL DESCRIPTION.

A volta label on a matra gives its row a bracket lane. Consecutive matras with one label share a bracket, typed BEGIN, MID, END or BEGIN_END as in VexFlow. Every ending but the last is closed. A tihai item expands in place into ordinary matras and its bracket runs from the first to the last note (figure 1.6-2).

1.6.2.1 Tihai Check.

A tihai is a phrase repeated N times with an optional gap. Its last note must land on sam, or on the matra the author names. Otherwise lowering reports tihai.sam.

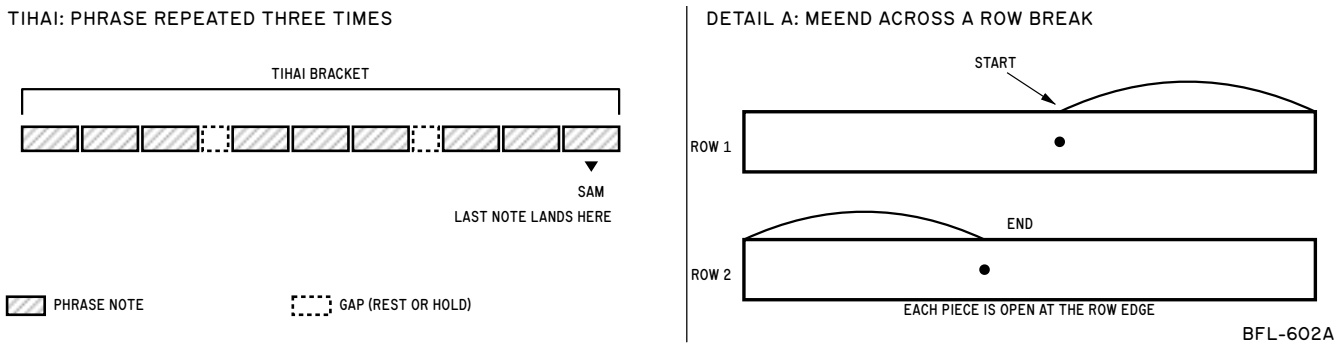


Figure 1.6-2. Tihai Bracket And Partial Meend

1.6.3 PERFORMANCE AND DESIGN DATA.

Table 1.6-1. Element Classes And Where They Sit

Class	Where it is drawn
grouping_curve	UNDER SWAR OR LYRIC CLUSTER
meend	ARC, MEEND LANE (12 PX)
tihai_bracket	TIHAI LANE (20 PX)
volta_bracket	VOLTA LANE (22 PX)
repeat_bar	VIBHAG BAR OR ROW EDGE
vibhag_line	BETWEEN VIBHAGS
line_end_bar	END OF A FULL CYCLE
khatka_bracket	BESIDE THE SWAR
taal_marker	MARKER LINE, UNDER A VIBHAG START
kan_swar	KAN LANE, ABOVE ITS SWAR
octave_dot	ABOVE (TAAR) OR BELOW (MANDRA)
section_heading	CENTRED ABOVE ITS ROWS

NOTE

A repeat sign sits only on a vibhag bar or at a row edge. Anywhere else it is ignored and repeat.offbar is reported.

WRITE A SECOND ENDING — **USER**

```
1 LINE - repeatStart: true
2 MATRAS - volta: 1 on ending 1
3 LAST - repeatAfter: true
4 MATRAS - volta: 2 on ending 2
  √ render - 2. sits under 1.
```

SECTION 1 · SUBSECTION 1.7
LABELS, BROWSER AND VERIFICATION

1.7.1 INTRODUCTION.

Every element opens an SVG group named after its BhatkhandeSet class and carrying its metadata. The context records the box of each group, so one render is also a labelled sample: one box per element, 19 classes, written as YOLO text and as JSON (figure 1.7-1).

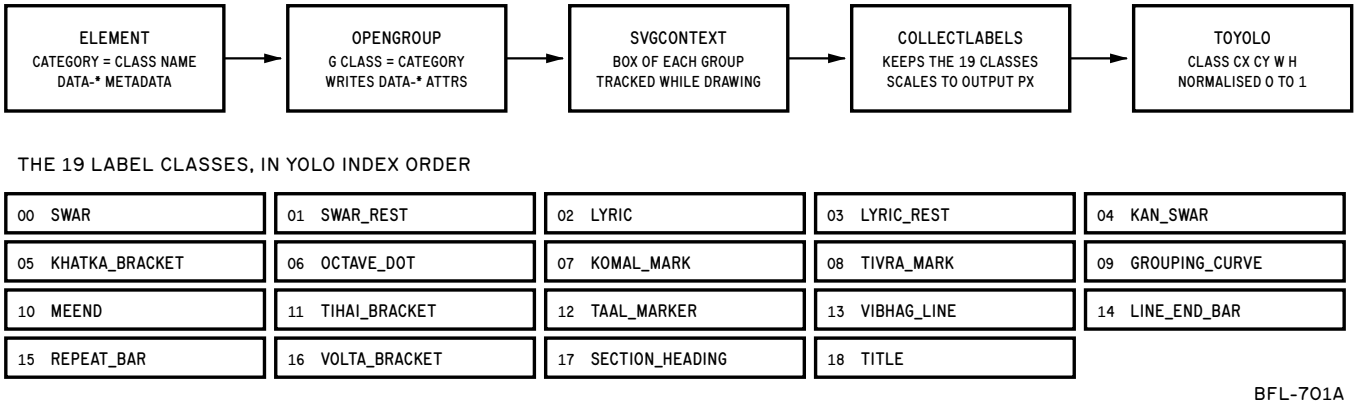


Figure 1.7-1. Label Export Chain And Class List

1.7.2 FUNCTIONAL DESCRIPTION.

collectLabels() keeps the groups whose class is in the taxonomy and scales their boxes to the output size. toYOLO() normalises them. The engine runs in Node and in the browser. The outline provider shapes Devanagari with fontkit and gives the same SVG in both; the canvas provider needs no font library (figure 1.7-2).

1.7.2.1 Open Items.

Meend is drawn as an arc above the swars and a silence as an empty slot, because no scanned page shows either. Paluskar notation is out of scope. Bihag geet 3 draws three tivra strokes against two annotated; some strokes on that page are faint.

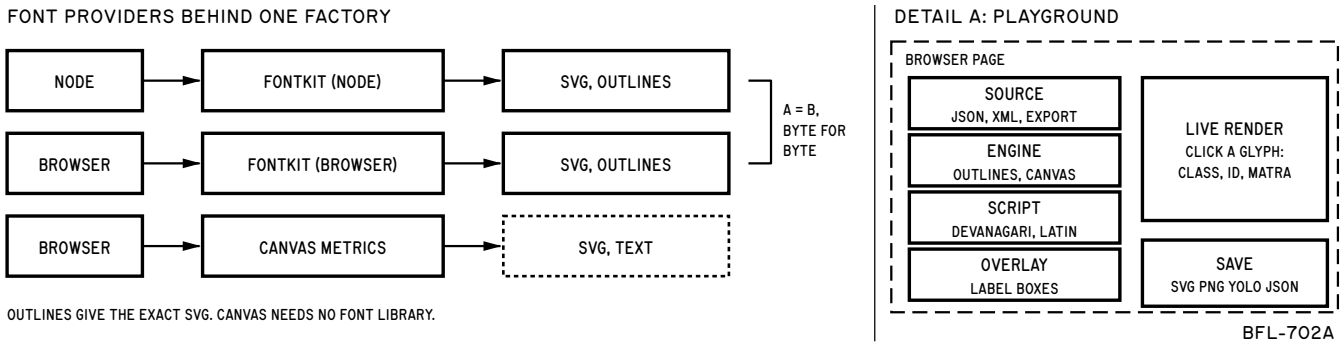


Figure 1.7-2. Font Providers And Playground

1.7.3 PERFORMANCE AND DESIGN DATA.

Table 1.7-1. Verification Record

Check	Result
Bihag geet 4 vs annotation	ALL CLASS COUNTS EQUAL
Bhimpalasi geet vs annotation	ALL CLASS COUNTS EQUAL
Alhaiya Bilawal dhamar	KAN, SWAR, LYRIC, DASH, BAR
Label Studio import, ektaal	ANNOTATED COUNTS REPRODUCED
Convention tests (8)	PASS
Browser SVG vs Node SVG	BYTE-IDENTICAL, 5 FILES
Every example	RENDERS WITH BOTH ENGINES
Browser round trip, export	MATCHES NODE
Playground smoke test	PASS, HEADLESS CHROMIUM
Bihag geet 3, tivra strokes	3 DRAWN, 2 ANNOTATED (OPEN)
Test suite	18 OF 18 PASS

NOTE

If port 5173 is busy, the playground server tries the next port and prints the address it opened.

START THE PLAYGROUND	USER

```
1 SHELL - npm run web
  √ address printed (5173 or next)
2 BROWSER - open the address
3 DROP - bandish, XML or export
4 TOGGLE - overlay label boxes
5 CLICK - inspect a swar
6 SAVE - svg, png, yolo, json
```